

Rapport:Créative Yumland



Projet Creative-Yumland

Pré-ing2 MI-2
Ben-abdesselam Youcef
Roussaux Zachary
Kiala Mputu Ronyx

Sommaire

I.Phase#1 (p.2)

- 1. Introduction**
- 2. Organisation des pages**
- 3. Adaptation aux différents utilisateurs**
- 4. Erreurs et solutions lors du codage**
- 5. Problèmes divers**

II.Phase#2 (p.6)

III.Phase#3 (p.8)

IV.Phase#4

I.Phase#1

1. Introduction

Le but de cette première étape était de fabriquer le visuel du site *Mosaïque Yum* en utilisant du HTML pour la structure et du CSS pour le design. L'idée est d'avoir des pages qui s'affichent correctement et sur lesquelles on peut cliquer.

Afin de pouvoir mener à bien notre projet nous avons pris la décision de nous répartir les tâches pour plus d'efficacité et d'autonomie. Nous nous sommes organisés de la manière suivantes:

- Ronyx s'est occupé de la du choix du modèle du site, de la charte graphique, de l'image de la marque (nom, logo, produit), ainsi que de la mise en place des pages d'accueil, de connexion, d'inscription, de la carte et de la partie .css associés.
- Youcef lui s'est chargé de la conception des pages de profil, d'administrateur, de notation, livreur, restaurateur. Ainsi que la partie .css qui leur est associée conformément à la charte graphique mise en place.
- Zachary s'est chargé de l'incorporation des différents produits sur le site, ainsi que de la vérification des potentielles erreurs anciennement présentes sur le code du site.

2. Organisation des pages

Pour éviter de nous y perdre et pour avoir un code propre, nous avons fait quelques choix importants :

- **Un seul fichier de style** : Toutes les couleurs, les polices et les mises en page de toutes les pages sont gérées par un seul et unique fichier `style.css`. C'est beaucoup plus facile à modifier que d'avoir du style éparpillé partout. (C'était aussi l'une des consignes de la phase.)
- **Séparer l'accueil et le menu complet** : Nous avons mis les promotions et les nouveautés sur la page d'accueil (`Accueil.html`), et avons créé une page à part pour toute la liste

des pizzas et des filtres de recherche ([Carte.html](#)). Ça évite d'avoir une première page trop chargée.

- **Un menu toujours présent** : Nous avons fait attention à mettre la barre de navigation sur absolument toutes les pages liée à l'utilisateur, même pour la page de connexion ou d'inscription. Comme ça, le visiteur ne se retrouve jamais coincé sur une page sans pouvoir revenir en arrière. Il peut même y retourner en appuyant sur le logo.

3. Adaptation aux différents utilisateurs

Nous avons dû concevoir des pages adaptées à ce que chaque personne vient faire sur le site :

- **Les clients et l'administrateur** : Leurs pages sont sur fond clair, avec des boutons jaunes ou rouges bien visibles pour facilement commander ou lire les tableaux de bord.
- **Les livreurs (sur téléphone)** : C'était la partie la plus difficile. La page du livreur a été pensée spécifiquement pour un écran de téléphone. Le fond est sombre, les textes sont plus gros, et surtout, nous avons codé des boutons d'action massifs qui prennent toute la largeur de l'écran pour qu'ils soient faciles à cliquer même avec des gants de scooter.

4. Erreurs rencontrées et corrigées lors du codage

Pendant la création du site, nous avons fait plusieurs erreurs d'apprentissage que Nous avons dû corriger :

- **Le CSS au mauvais endroit** : Au début, pour aller plus vite, Ronyx avait mis des codes de couleur et de taille directement dans mes lignes HTML (en utilisant `style="..."`). J'ai compris que ça rendait le code illisible. J'ai donc tout effacé pour créer des classes propres dans le fichier `style.css`.
- **Les boutons dans les liens** : Sur mes premières pages, j'avais mis des balises de bouton (`<button>`) à l'intérieur de balises de liens (`<a>`). C'est une erreur de code qui peut faire bugger le site. J'ai par ailleurs appris cela en demandant à une IA de vérifier les

eventuelles erreurs dans mon code. J'ai donc retiré les `<button>` et j'ai directement donné l'apparence d'un bouton à mes liens grâce au CSS.

- **Le problème des fausses informations** : Sachant qu'il n'y a pas encore de base de données, nous avons dû inventer de fausses commandes ou de faux profils (comme "Jean Dupont") directement dans le HTML. Le but était juste de vérifier que nps tableaux s'affichent bien

5.Problèmes divers rencontrés

Nous avons également fait face à quelques problèmes d'organisation qui ont ralenti notre progression durant cette phase :

- **La communication dans le groupe** : Pour diverses raisons, nous n'avons pas toujours pu assister tous ensemble aux séances de TD. Nous avons donc dû nous organiser principalement via les réseaux sociaux. Le problème, c'est que cette méthode a parfois créé de la confusion. Les temps d'attente pour obtenir les réponses de chacun ont ralenti le développement du projet.
- **Les imprévus personnels** : L'avancement du projet a aussi été freiné par des problèmes personnels au sein du groupe. Par exemple, Youcef a malheureusement rencontré des soucis familiaux puis est tombé malade, ce qui l'a empêché de travailler avec nous pendant une certaine période. Nous avons donc dû nous réadapter pour compenser son absence temporaire et continuer à avancer.

II.Phase#2

1. Organisation du code et des fichiers

Pour cette Phase 2, on a dû complètement repenser notre façon d'organiser nos fichiers. Au lieu de copier-coller le même code HTML sur toutes nos pages, on a découpé le site.

- On a créé un fichier header.php et un footer.php qu'on inclut partout avec la fonction include(). Comme ça, si on veut modifier le menu, on ne le fait qu'à un seul endroit.
- On a aussi créé un fichier fonctions.php. Son but est de ranger toutes les fonctions qui lisent ou écrivent dans nos fichiers JSON. Ça permet de garder nos pages d'affichage beaucoup plus propres et de séparer le "visuel" de la "logique".

2. Base de données (JSON) et Connexion des utilisateurs

Pour stocker nos données (les comptes clients, le menu du restaurant), on a utilisé des fichiers .json. En PHP, on utilise les fonctions json_decode() pour lire ces fichiers et les transformer en tableaux que l'on peut manipuler, et json_encode() pour sauvegarder les nouveaux inscrits.

Le gros morceau de cette phase a été le système de connexion. Pour que le site se souvienne de qui est connecté, on a utilisé les sessions (\$_SESSION).

- Quand un utilisateur se connecte sur Login.php, on vérifie si son email et son mot de passe existent dans notre JSON.
- Si c'est bon, on crée une session et on garde en mémoire son "rôle" (client, admin, livreur...).
- Grâce à ça, notre header.php s'adapte tout seul avec des if/else. Si un client se connecte, il voit son panier. Si c'est un livreur, il voit sa tournée.

3. Le panier et le système de paiement (Pôle 3)

Toute la partie e-commerce a été développée pour que le client puisse passer une vraie commande.

- **Le Panier** : Les articles choisis sont sauvegardés dans un tableau de session \$_SESSION['panier'], ce qui permet de naviguer sur le site sans perdre ses choix.
- **Le Paiement** : Sur la page Paiement.php, on calcule le total et on se connecte à la plateforme Cybank. Pour sécuriser la transaction,

on utilise un fichier getapikey.php et la fonction md5() pour créer un code de contrôle unique. Cela empêche de modifier le prix dans l'URL.

- **La validation** : Quand la banque renvoie le client sur RetourPaiement.php, on vérifie que le paiement est bien "accepted" et que le code de sécurité est le bon. Si tout est ok, on enregistre la commande dans le JSON et on vide le panier.

4. Les problèmes rencontrés et le débogage

Passer du HTML au PHP ne s'est pas fait tout seul et on a eu pas mal de bugs à régler :

- **Les formulaires qui tournent dans le vide** : Au début, l'inscription ne marchait pas. On a dû corriger les méthodes d'envoi (POST) et surtout rajouter les attributs name sur nos balises <input> car PHP n'arrivait pas à récupérer les données tapées.
- **Le bug des redirections** : On a compris à nos dépens qu'en PHP, si on veut utiliser header("Location: ...") pour rediriger quelqu'un, on doit le faire tout en haut du fichier, avant d'afficher le moindre code HTML, sinon le serveur plante.
- **Erreurs de syntaxe** : Lors de l'intégration du paiement, on a eu des pages blanches à cause d'un simple point-virgule oublié à la fin d'une ligne ou d'un formatage de prix mal géré pour la banque.
- **Les galères avec Git** : Quand on a voulu mettre le travail des deux pôles en commun sur GitHub, on a eu des gros conflits car on modifiait les mêmes fichiers. On a dû utiliser les commandes du terminal (comme git pull --rebase) pour forcer la synchronisation proprement sans écraser le code de l'autre.

5. Organisation de l'équipe et imprévus

Nous avons eu un gros imprévu au début de cette phase : Youcef qui devait s'occuper du Pôle 2 (Authentification et Sessions) nous a lâchés. Pour que le projet ne prenne pas de retard, on s'est réorganisés. Ronyx a pris en charge la totalité de son pôle en plus de sa partie sur les JSON et l'architecture globale. Comme il avait créé la structure des données, c'était plus logique qu'il fasse aussi la connexion. Cela a permis à Zachary de se concentrer à 100% sur la création du panier et la connexion compliquée avec la banque (Pôle 3). Finalement, cette organisation nous a permis de tout finir dans les temps et d'avoir un site qui fonctionne de bout en bout.

III. Phase #3

L'objectif principal de cette troisième phase était de rendre le site interactif et fluide en réduisant au maximum les rechargements de page. Pour cela, nous avons dû intégrer massivement du JavaScript et utiliser l'API Fetch pour effectuer des requêtes asynchrones vers notre serveur. L'enjeu était de séparer clairement l'affichage (Front-end) de la logique de traitement des données (Back-end).

1. Ergonomie, Thème Dynamique et Sécurité

Pour améliorer l'expérience utilisateur, plusieurs fonctionnalités ont été développées côté client en manipulant le DOM :

- Le Mode Sombre : Nous avons mis en place un système de variables CSS (`:root`) modifiables via JavaScript pour basculer entre un thème clair et un thème sombre. Pour que le site mémorise ce choix à chaque visite, la préférence de l'utilisateur est sauvegardée dans un Cookie.
- Sécurité et confort des formulaires : Sur les pages de connexion et d'inscription, des scripts JavaScript vérifient la saisie en temps réel. Nous avons ajouté un bouton permettant d'afficher ou de masquer le mot de passe (icône œil) , ainsi qu'un compteur de caractères dynamique (qui change de couleur si la saisie est trop courte ou trop longue) pour les champs limités.

2. Le Catalogue et le Panier Asynchrones

C'est le cœur technique de cette phase. La page de la carte (`Carte.php`) ne se recharge plus jamais lors de la navigation :

- Filtres et Tris : Le script `catalogue.js` fait appel à une API (`api_get_plats.php`) qui lui renvoie l'intégralité du menu au format JSON. Les filtres par catégories se font de manière dynamique, et les tris (par prix croissant ou décroissant) sont effectués localement en JavaScript (avec la méthode `.sort()`) sur les données déjà chargées, sans refaire de requête au serveur.

- Le "Menu Builder" (Fenêtre Modale) : Pour la composition des menus, nous avons créé une interface modale générée entièrement en JavaScript. Le client peut choisir ses articles (pizza, boisson, dessert) via des listes déroulantes (`<select>`) créées dynamiquement en fonction du menu cliqué.
- L'ajout au panier : Le formulaire classique a été remplacé par une requête asynchrone (`fetch`) vers `api_add_cart.php`. En cas de succès, une micro-interaction visuelle modifie le bouton et une notification flottante (Toast) confirme l'ajout au client.

3. Les Dashboards Métiers en Temps Réel

Pour les profils spécifiques, les actions ont été fluidifiées :

- Administrateur : Il peut désormais bloquer ou débloquer un utilisateur d'un simple clic. La requête part en arrière-plan et l'interface se met à jour immédiatement (changement de couleur du bouton et du statut).
- Restaurateur et Livreur : Les changements d'états des commandes (passage de "En attente" à "En préparation", ou "Livrée") sont gérés par des requêtes asynchrones, permettant aux équipes de travailler sur une interface réactive.

4. Problèmes rencontrés et Débogage

Le passage à une architecture asynchrone a soulevé de nouveaux défis techniques :

- Le crash des requêtes JSON : Lors de la mise en place du catalogue asynchrone, notre JavaScript plantait (erreur de syntaxe). Nous avons découvert que notre fichier PHP `api_get_plats.php` renvoyait des erreurs HTML (dus à un mauvais chemin de fichier) au lieu d'un flux JSON propre. Il a fallu utiliser l'onglet "Réseau" des outils de développement du navigateur pour comprendre que le serveur ne trouvait plus `carte.json`.
- Les erreurs de nommage : Un simple "s" manquant dans l'appel d'un fichier script (`formulaire.js` au lieu de `formulaires.js`) a suffi à casser toute la logique d'interface d'une page. Cela nous a appris la rigueur absolue requise dans le nommage des fichiers liés.

- Les URLs de retour bancaire : Lors des tests de bout en bout, les commandes payées ne s'affichaient pas chez le restaurateur. Le problème venait de l'URL de retour de la banque (CYBank) qui pointait encore vers l'ancien dossier de la Phase 2. Nous avons corrigé cela en générant dynamiquement l'URL de retour en PHP (`$_SERVER['HTTP_HOST']`).
- Rendre les tableaux de bord interactifs a été le plus grand obstacle. Pour que l'interface se mette à jour toute seule, l'IA m'a recommandé d'utiliser la technique du "Polling" (`setInterval` en JS) qui interroge le serveur en boucle. Cela a généré des bugs invisibles très complexes à résoudre : par exemple, un simple chemin de fichier incomplet (`data/`) a fait crasher tout mon JavaScript en arrière-plan sans aucun message d'erreur. Cette épreuve m'a forcé à apprendre à déboguer rigoureusement via l'onglet "Réseau" du navigateur.

5. Organisation de l'équipe, travail en solo et aide de l'IA

La Phase 2 avait déjà été compliquée avec l'absence de Youcef, et la Phase 3 a connu un problème similaire. En plein milieu du travail, Zachary a rencontré des problèmes personnels qui l'ont empêché de participer pendant une grande partie de la phase. Je me suis donc retrouvé seul pour avancer.

Normalement, le travail était bien séparé : Zachary devait s'occuper de l'interface, du mode sombre et de la vérification des formulaires. De mon côté, je devais gérer les requêtes asynchrones, le catalogue et les tableaux de bord. Pour ne pas bloquer le projet, j'ai dû commencer à coder une bonne partie de ses tâches moi-même. Heureusement, Zachary est revenu sur la fin pour m'expliquer sa situation et faire quelques ajustements sur le code, mais le gros du développement a dû se faire dans l'urgence.

Comme j'étais seul pendant la majorité du temps et que le délai était court, j'ai décidé d'utiliser l'Intelligence Artificielle pour m'aider. Je m'en suis servi comme d'un outil pour corriger mes bugs, par exemple pour trouver plus vite les erreurs de syntaxe dans mon code JavaScript ou pour vérifier si mes requêtes Fetch étaient bien écrites. Ça m'a fait

gagner un temps précieux et ça m'a permis d'avancer malgré la situation.

Forcément, avec ce contretemps et tout ce travail imprévu, nous n'avons pas eu le temps de faire la fonctionnalité bonus qui permettait de modifier une commande après l'avoir payée. Nous avons préféré nous concentrer sur les fonctionnalités obligatoires (le catalogue asynchrone, le panier, la sécurité) pour être sûrs de rendre un site qui marche parfaitement et sans bug pour la date limite.